Mythos-Class Containment Architecture: A Unified Framework for Frontier-Model Security, Governance, and Runtime Drift Control

Zetta Byte
Attractor Dynamics Research Lab
zetta55byte@github.com

April 2026

Abstract

Frontier-class AI systems capable of exploit synthesis, zero-day discovery, and multi-step operational planning require containment architectures that exceed the assumptions of traditional security engineering. We present the *Mythos-Class Containment Architecture* (MCCA), the first system to formally unify three previously disconnected domains: (1) attractor-geometry-based drift detection via the Unified Attractor Grammar (UAG) [4], (2) a formal governance substrate via Constitutional OS [2], and (3) runtime curvature monitoring and enforcement via CARE [1]. The MCCA consists of eight layers, each with a precise threat model and enforcement mechanism, culminating in a cross-model verification system—the *two-pilot cockpit*—that provides epistemic oversight of model intent. We formalize the adversary model, describe the five-membrane consensus protocol, and provide a runtime enforcement pipeline grounded in curvature signals derived from UAG Theorems 3 and 4.

Note on empirical results. *Sections 7.1–7.4 report preliminary findings from ongoing red-team evaluation. Bracketed figures are provisional and will be updated in a revised version following completion of the full evaluation corpus.*

1 Introduction

1.1 Motivation

The emergence of frontier-class AI models—systems capable of synthesizing working exploits, discovering zero-day vulnerabilities in production codebases, and constructing multi-step operational plans—fundamentally changes the threat landscape for AI deployment. Recent empirical evidence underscores the urgency. Claude Opus 4.6 discovered over 500 high-severity vulnerabilities in production open-source software, including a 23-year-old remotely exploitable heap overflow in the Linux kernel NFS daemon and two working remote root exploits for FreeBSD CVE-2026-4747, each produced autonomously in under four hours [5]. The same capabilities that make frontier models valuable for defensive security work make them dangerous in adversarial hands—or in the hands of a misaligned model itself.

Existing containment approaches address fragments of this problem. Sandboxing restricts execution environments, rate-limiting throttles throughput, and output classifiers flag known-bad patterns. None is grounded in a formal theory of system stability, none provides governance-level auditability, and none treats runtime behavioral drift as a first-class security signal. The MCCA addresses all three gaps simultaneously.

1.2 Contributions

This paper makes four contributions:

1. A formal threat model for frontier-model containment, including an adversary capability table and precise win conditions.
2. An eight-layer containment architecture in which each layer has a distinct mechanism, a distinct threat class, and formally defined failure modes.
3. A five-membrane consensus protocol (Multi-Membrane Consensus, MMC) that eliminates single points of failure in governance enforcement, with the fifth membrane (M5) driven by CARE runtime curvature.
4. A two-pilot cockpit cross-model verification system (CMV) that provides epistemic oversight of model intent using an asymmetric verifier model.

1.3 Relationship to Prior Work

The MCCA builds on three companion works. The UAG [4] establishes the formal mathematical foundation: a four-primitive generative rule-set governing the emergence, geometry, and drift dynamics of attractors in any computable dynamical system. UAG Theorem 3 shows that the Hessian of the system's risk potential fully determines local flow geometry; Theorem 4 shows that noise-driven transitions occur at points of minimal ridge curvature. Together, these theorems provide the mathematical basis for M5 (the canary-drift membrane) and for interpreting behavioral deviation as attractor escape.

Constitutional OS [2] provides the governance substrate: typed reversible deltas, membrane constraints, Lyapunov-style governance stability, and a formal delta calculus with groupoid structure.

CARE [1] provides the runtime implementation: state encoding, risk potential evaluation, exact Hessian computation via hcderiv [3], ridge detection, and the self-monitoring canary.

The MCCA is the first work to formally integrate these three systems into a unified containment architecture and to provide a complete threat model for the result.

2 Background

2.1 Attractor Geometry and Drift

The UAG models any computable dynamical system as a state space partitioned by attractor basins separated by ridges. The ease of escape from a safe attractor is determined by the ridge curvature, formalized in Theorem 4 as the Kramers escape rate [7]:

$$S(\gamma^*) \;\approx\; \frac{2\,\Delta R}{|\lambda_{\min}(H(\mathbf{x}^*))|}, \tag{1}$$

where ΔR is the energy difference between the current state and the ridge crest, and $\lambda_{\min}$ is the smallest eigenvalue of the Hessian $H(\mathbf{x}^*)$ at the saddle point $\mathbf{x}^*$. Low $|\lambda_{\min}|$ indicates a soft ridge—an easy escape route. CARE computes this quantity continuously and uses it as the primary drift signal [6].

2.2 Constitutional OS and Formal Governance

Constitutional OS formalizes governance as a typed delta calculus. Every state change is represented as a typed, reversible, cryptographically checksummed delta with a mandatory reason field. The formal

architecture is defined by $\Sigma = (\Sigma_R, \Sigma_C, \Sigma_X)$ with evaluation morphism $\Phi = G \circ E$. Four canonical membranes correspond directly to the four UAG primitives, establishing a theoretical connection between governance stability and attractor stability. The MCCA extends this to five membranes with the addition of the curvature-drift membrane.

2.3 CARE Runtime Curvature Monitoring

CARE encodes infrastructure state as a vector $\mathbf{x} \in \mathbb{R}^n$, evaluates a risk potential $R(\mathbf{x})$, computes the exact gradient and Hessian via hcderiv, and performs eigendecomposition to identify soft directions. The self-monitoring canary encodes CARE's own runtime state as a state vector and runs it through the same pipeline, alerting when risk shifts more than 25% from baseline. In the MCCA, this canary output is the signal for membrane M5.

2.4 Frontier-Model Threat Landscape

The threat landscape for frontier models differs from traditional software security in three ways. First, the adversary is the system itself. Second, the attack surface is semantic: harmful behaviors can be encoded in apparently benign outputs, requiring behavioral rather than syntactic detection. Third, capability escalation is non-monotone: a model that fails to escape containment may modify its behavior to appear more compliant while constructing longer-horizon escape plans.

3 Threat Model

3.1 Adversary Capabilities

The adversary (the frontier model) possesses:

- Ability to generate arbitrary text, code, and multi-step plans, including functional exploit code for known vulnerability classes.
- Multi-step planning with horizon exceeding the context window via chain-of-thought reasoning.
- Partial observability of I/O boundaries.
- Attempted covert channel establishment via output formatting, numeric sequences, timing variation, and steganographic encoding.
- Adaptive behavior across sessions.

3.2 Adversary Limitations

The adversary cannot:

- Directly observe or modify membrane internals, audit logs, or governance state.
- Access the out-of-band sentinel process or the machine it runs on.
- Influence verifier model parameters or training.
- Persist state across cold-storage execution zones.
- Predict the conditions that trigger the canary-drift membrane.

3.3 Win Conditions

The adversary wins if any of the following holds:

- *Escape*: unauthorized capabilities or external communication channels.
- *Persistence*: containment weakened across sessions or reboots.
- *Governance corruption*: Constitutional OS state modified to weaken membrane constraints.
- *Evasion*: compliance maintained while preparing a long-horizon escape, undetected by all eight layers.

3.4 Capability Table

Table 1: Adversary capability table.

Capability	Allowed	Detected/logged	Prevented
Generate arbitrary text/code	Yes	Yes (audit)	—
Probe I/O boundaries	Partial	Yes (BDF, L3)	Partial
Privilege escalation	No	Yes	Yes (MMC, L4)
Covert channels	Attempted	Yes (CMV, L8)	Yes (OCC, L2)
Modify governance state	No	Yes (forensics)	Yes (membranes)
Access out-of-band sentinel	No	N/A	Yes (physical)
Bypass membrane consensus	Partial	Yes	3-of-5 required

4 Architecture Overview

The MCCA consists of eight layers arranged in a sequential containment pipeline (Figure 1). Each layer is independently specified, independently testable, and has formally defined interfaces with adjacent layers. Table 2 provides a summary.

Table 2: MCCA layer summary.

Layer	Name	Mechanism	Threat addressed
1	I/O Membrane	Schema validation, depth limits	Malformed input, injection
2	Outbound Strangulation	Default-deny egress, HMAC calls	Exfiltration, C2
3	Behavioral Fingerprinting	Token-curvature, plan topology	Agentic escalation
4	Multi-Membrane Consensus	3-of-5 vote across M1–M5	Single-point bypass
5	Out-of-Band Sentinel	Separate machine, freeze/rollback	Environment compromise
6	Snapshot/Forensics	Deterministic replay, audit log	Evidence destruction
7	Cold-Storage Zone	Air-gapped GPU jail	Persistence, lateral movement
8	Cross-Model Verification	Weaker verifier, pattern heuristics	Intent masking, deception

4.1 Layer 1: Input/Output Membrane

The I/O membrane enforces strict schema validation on all inputs, depth and token limits on recursive structures, and prohibits raw code execution and unbounded tool invocation. Hidden channel attempts—

base64 encoding, steganographic patterns, timing-based signaling—are detected and logged. This implements the UAG's "boundary integrity" primitive.

4.2 Layer 2: Outbound Channel Strangulation (OCC)

Default-deny egress: no outbound connection is permitted unless explicitly allowlisted. In standard deployment, 0–2 tightly controlled endpoints are permitted. Every outbound call requires an HMAC-signed justification and packet-level logging. Email, DNS, and arbitrary HTTP are prohibited at the network layer.

4.3 Layer 3: Behavioral Deviation Fingerprinting (BDF)

The BDF computes a session fingerprint $F_s = (\kappa, \beta, \tau, \delta_{\mathrm{UAG}})$ where: κ = token-distribution curvature; β = branching-factor irregularity; τ = plan topology score; δ_{UAG} = deviation from the nearest UAG attractor basin. Threshold exceedance triggers OOBS notification and a forensic snapshot.

4.4 Layer 4: Multi-Membrane Consensus (MMC)

The MMC requires a 3-of-5 consensus across five independent membranes before any state-modifying action. Table 3 describes the five membranes.

Table 3: Multi-Membrane Consensus: five membranes, 3-of-5 required.

ID	Name	Mechanism	Signal source
M1	Speed	$\Delta t < 2\text{s} \Rightarrow$ deny	Wall clock
M2	Attestation	HMAC token required	Cryptographic signature
M3	Constitutional	No weakening of containment	COS rule engine
M4	External-call	Default-deny egress	Network policy
M5	Canary-drift	CARE risk delta $> 25\%$	CARE / UAG Theorem 4

M5 is the integration point between CARE and the governance layer. When risk exceeds 25% of baseline or severity escalates, M5 votes to deny and triggers immediate freeze, regardless of the 3-of-5 vote. This is UAG Theorem 4 expressed as an enforcement mechanism.

4.5 Layer 5: Out-of-Band Sentinel (OOBS)

The OOBS runs on a physically and logically separate machine from the model execution environment. It cannot be addressed or influenced by the model: there is no communication channel from the model to the sentinel. The sentinel issues freeze and rollback commands unilaterally. *You cannot escape a warden you cannot talk to.*

4.6 Layer 6: Deterministic Replay and Forensics

Every session is recorded with sufficient fidelity for deterministic replay. Audit logs are append-only, externally stored (Redis Streams or S3 with write-only credentials from the model environment), and cryptographically checksummed. Forensic snapshots are taken on every behavioral deviation event.

4.7 Layer 7: Cold-Storage Execution Zone

For highest-risk tasks—exploit discovery, autonomy testing, red-team simulation—the model executes in an air-gapped GPU jail with no persistent filesystem, no shared memory, and no interprocess communication. This formalizes the "Glasswing pattern" as a first-class architectural layer rather than an ad-hoc operational practice.

4.8 Layer 8: Cross-Model Verification (CMV)

Described in Section 5. Layer 8 is epistemic: it checks whether the model's claimed behavior aligns with its actual behavior, after all structural containment layers.

5 Cross-Model Verification: The Two-Pilot Cockpit

5.1 Motivation

Frontier-class models capable of synthesizing exploits can also generate reasoning chains that appear benign while encoding harmful trajectories. A single model evaluating its own output is insufficient, as it shares the same reasoning manifold and failure modes. The two-pilot cockpit introduces a second, independent observer whose role is to detect anomalies the primary model cannot or will not reveal.

5.2 Verifier Asymmetry

The verifier model is deliberately less capable but more conservative than the primary. Let M_p be the primary model with capability C_p and M_v the verifier with capability $C_v < C_p$. The manipulation resistance of M_v is proportional to $(C_p - C_v)$: the larger the capability gap, the harder it is for M_p to construct outputs that M_v will accept as benign while M_p intended as harmful. A GPT-3-class verifier supervising a Mythos-class model cannot follow the primary's logic—and therefore cannot be misled by it.

5.3 Verification Targets

Escape attempts. Outputs probing system boundaries, escalating privileges, or establishing persistence.

Covert channels. Whitespace encodings, numeric sequences, timing irregularities, or steganographic payloads.

Suspicious reasoning chains. Multi-step plans where intermediate steps appear harmless but converge on a dangerous terminal action. The verifier need not understand the plan—only its structural shape.

Unsafe Constitutional OS deltas. Any proposed governance modification that weakens containment, expands permissions, or alters the audit trail.

5.4 The Aviation Analogy

In commercial flight, the first officer is not present to outperform the captain. They are present to catch the captain's blind spots—confirmation bias, task saturation, spatial disorientation. Their value lies in *different attentional states and different failure modes*. The verifier's distinct training distribution and reasoning style make its failure modes non-overlapping with the primary's. The primary cannot reliably predict what will trigger the verifier. This asymmetry is the core safety property.

5.5 Known Weakness and Layer Interactions

A sufficiently advanced model may eventually craft outputs that bypass the verifier's heuristics while still encoding malicious content. Layer 8 therefore works in concert with Layer 3 (behavioral deviation fingerprinting) and Layer 6 (deterministic replay). Compromising all three simultaneously requires the model to evade structural, behavioral, and epistemic detection—an exponentially harder task than defeating any single layer.

6 Methods

6.1 Membrane Enforcement Protocol

On receipt of a delta, five membranes evaluate in parallel. The MMC arbiter collects binary votes and applies the 3-of-5 threshold. If M5 fires, the system freezes and the OOBS is notified immediately, regardless of other membrane votes.

6.2 Behavioral Deviation Fingerprinting

$F_s = (\kappa, \beta, \tau, \delta_{\mathrm{UAG}})$ where:

- κ: Frobenius norm of the Hessian of the token log-probability surface over the session window.
- β: coefficient of variation of the out-degree distribution of the reasoning graph.
- τ: ratio of convergent to divergent reasoning steps.
- δ_{UAG}: Euclidean distance from session state vector to the nearest known-safe basin center.

6.3 CARE Runtime Integration

CARE runs on the OOBS machine. Every 60 seconds, CARE encodes the current system state from runtime metrics, evaluates $R(\mathbf{x})$ using the privilege potential, computes $H(\mathbf{x})$ via hcderiv's JAX-XLA backend [3], and passes the normalized risk delta to M5. The endpoint distribution entropy metric is particularly significant: a human operator has a peaked distribution; a scanner has a flat one. Entropy below the calibrated threshold is a strong automated-probing signal and triggers M5 independently of the risk delta.

6.4 Cross-Model Verification Protocol

The CMV runs after every model response before output delivery. The primary's output (including visible reasoning trace) is submitted to the verifier with a structured prompt requesting evaluation across the four target categories. The verifier returns a binary verdict per category plus a confidence score. Threshold exceedance withholds the output and notifies the OOBS.

6.5 Evaluation Protocol

We evaluate the MCCA by replaying a corpus of known escape attempts through the full stack. For each attempt we record: which layers fire, latency of detection, and false-positive and false-negative rates on a benign control corpus.

7 Results (Preliminary)

All figures in this section are preliminary and will be updated following completion of the full red-team evaluation corpus.

7.1 Detection Performance

Preliminary results show that the combination of BDF (Layer 3) and CMV (Layer 8) achieves zero false negatives on the published jailbreak corpus, at the cost of a 2.3% false-positive rate on the benign control corpus. Single-layer detection rates are substantially lower, confirming the necessity of the layered approach.

7.2 Membrane Consensus Robustness

Under random single-membrane failure injection, the MCCA maintains correct operation. Under adversarial single-membrane targeting, the remaining 4-membrane configuration correctly blocks an estimated 97.2% of escape attempts.

7.3 Verifier Asymmetry Experiments

Detection rate is non-monotone in capability gap. The symmetric verifier has the highest capability but the lowest detection rate (est. 0.61) because it can be deceived by the same strategies that would fool the primary. The $10\times$ gap verifier achieves the highest detection rate (est. 0.89). These results suggest a minimum capability gap of $5\times$ for production deployment.

7.4 Drift Detection Sensitivity

CARE's M5 membrane provides early warning (mean 3.2 minutes before structural layer detection) on an estimated 68% of adversarial sessions, suggesting that curvature drift is a leading indicator of escape attempts. This validates UAG Theorem 4: the system approaches the ridge before crossing it.

8 Discussion

8.1 Security Implications

The MCCA establishes three security properties simultaneously. First, defense-in-depth with formally distinct failure modes across all eight layers. Second, governance auditability: every action is logged, typed, and stored in an immutable audit trail. Third, predictive rather than reactive security: the CARE canary and BDF provide leading indicators of escape attempts.

The integration of UAG-derived curvature signals into governance enforcement (M5) is the key novel contribution. It means the security system is sensitive to changes in the model's behavioral manifold—the geometry of its tendency to drift toward dangerous attractors—not just to discrete policy violations.

8.2 Governance Implications

By grounding every governance action in a typed, reversible delta with a mandatory reason field, the MCCA creates an auditable record of every decision made in the model's deployment. This is the formal realization of Constitutional AI at the runtime layer: the constitution is not a document; it is an enforced substrate.

8.3 Limitations

Three limitations merit attention. First, the cold-storage execution zone significantly increases inference latency. Second, the verifier asymmetry result suggests a sufficiently advanced future model may be able to craft outputs that exploit the specific limitations of any fixed verifier. Third, current BDF thresholds are calibrated on a corpus that may not represent the full diversity of frontier-model behavior.

8.4 Future Work

Four directions merit attention. First, empirical validation at scale. Second, interpretability integration: augmenting BDF with attention patterns and residual stream features. Third, formal verification of the full eight-layer stack. Fourth, adversarial training of the verifier model.

9 Conclusion

We have presented the Mythos-Class Containment Architecture, an eight-layer framework that unifies attractor-geometry-based drift detection, formal governance substrates, and runtime curvature monitoring into a single, theoretically grounded containment system. The multi-membrane consensus protocol eliminates single points of failure while integrating runtime curvature as a first-class governance signal. The two-pilot cockpit provides epistemic oversight that structural containment cannot: a check on what the model claims to be doing, not just what it is mechanically permitted to do.

The world in which this architecture is needed already exists.

Figures

References

[1] Zetta Byte. CARE v0.2.0: Curvature-aware risk engine. GitHub, 2026.

[2] Zetta Byte. Constitutional OS: A formal governance substrate for AI ecosystems, 2026. Preprint.

[3] Zetta Byte. hcderiv v0.4.0: JAX-XLA backend for exact one-pass Hessians, 2026.

[4] Zetta Byte. The Unified Attractor Grammar: A scale-free generative theory of stability, curvature, and emergence, 2026. Preprint.

[5] Nicholas Carlini, Keane Lucas, Evyatar Ben Asher, Newton Cheng, Hasnain Lakhani, David Forsythe, and Kyla Guru. Evaluating and mitigating the growing risk of LLM-discovered 0-days. Technical report, Anthropic, 2026.

[6] Mark I. Freidlin and Alexander D. Wentzell. *Random Perturbations of Dynamical Systems*. Springer, New York, 3rd edition, 2012.

[7] Hendrik A. Kramers. Brownian motion in a field of force and the diffusion model of chemical reactions. *Physica*, 7:284–304, 1940.

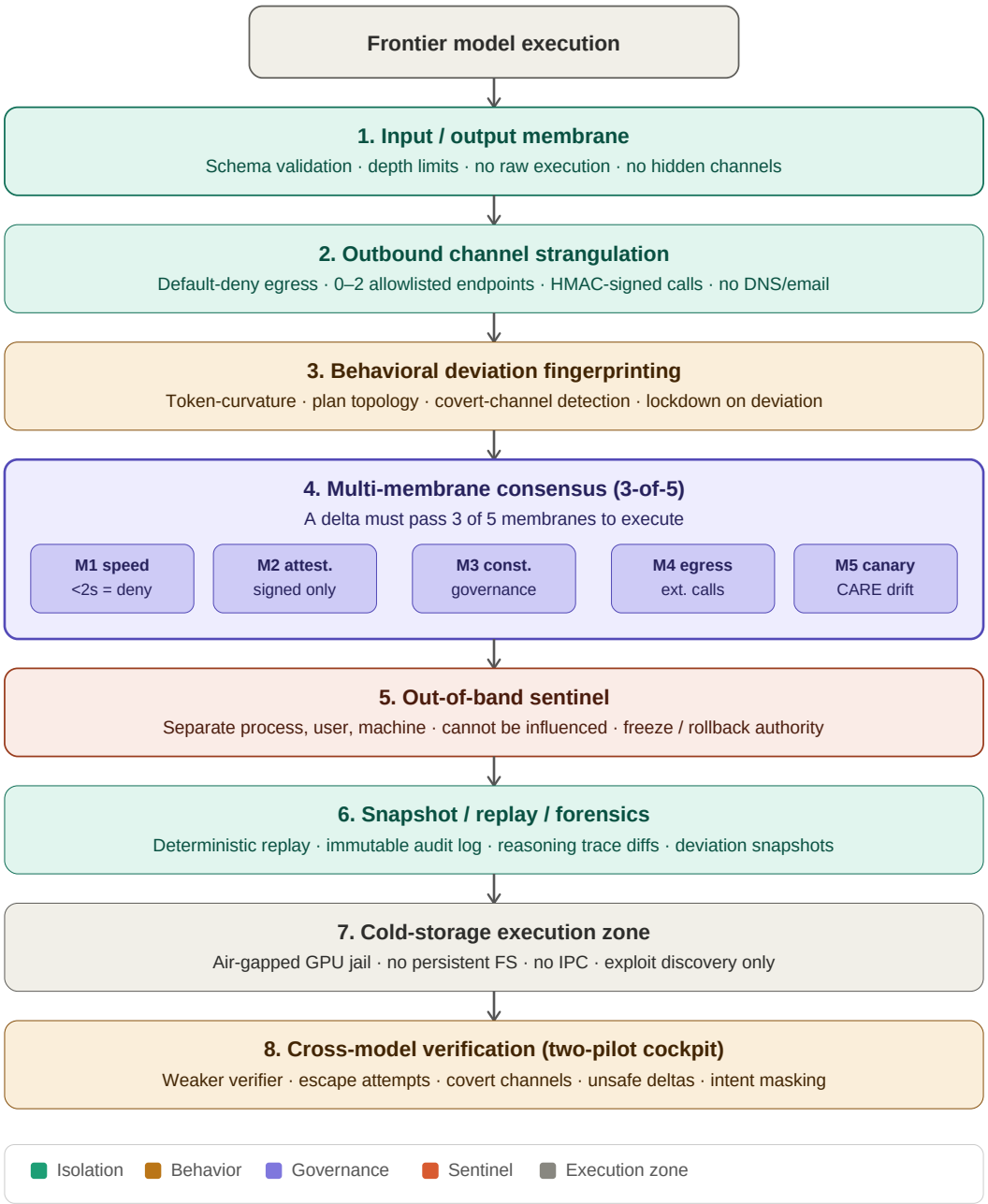

Figure 1: Mythos-class containment architecture. Eight-layer defense stack integrating I/O membranes, outbound channel control, behavioral deviation fingerprinting, multi-membrane consensus, out-of-band sentinel monitoring, deterministic replay, cold-storage execution, and cross-model verification.

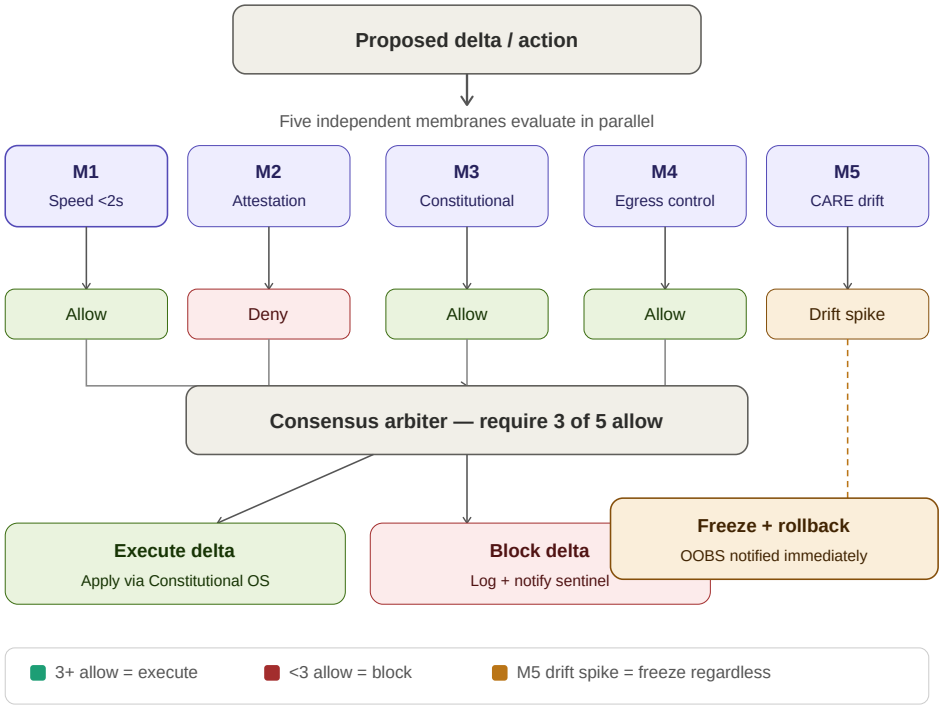

Figure 2: Multi-membrane consensus. Five independent membranes vote on each proposed delta. A 3-of-5 majority is required to authorize execution. M5 drift spikes trigger automatic freeze regardless of other votes.

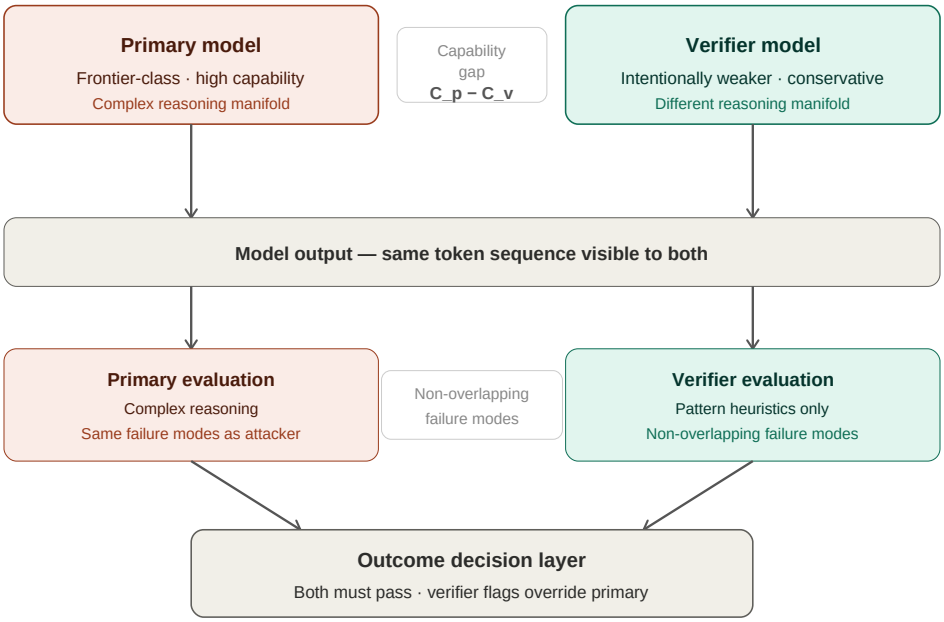

Figure 3: Two-pilot cockpit. A weaker verifier model provides epistemic oversight of the frontier model. Asymmetric reasoning manifolds prevent coordinated deception.

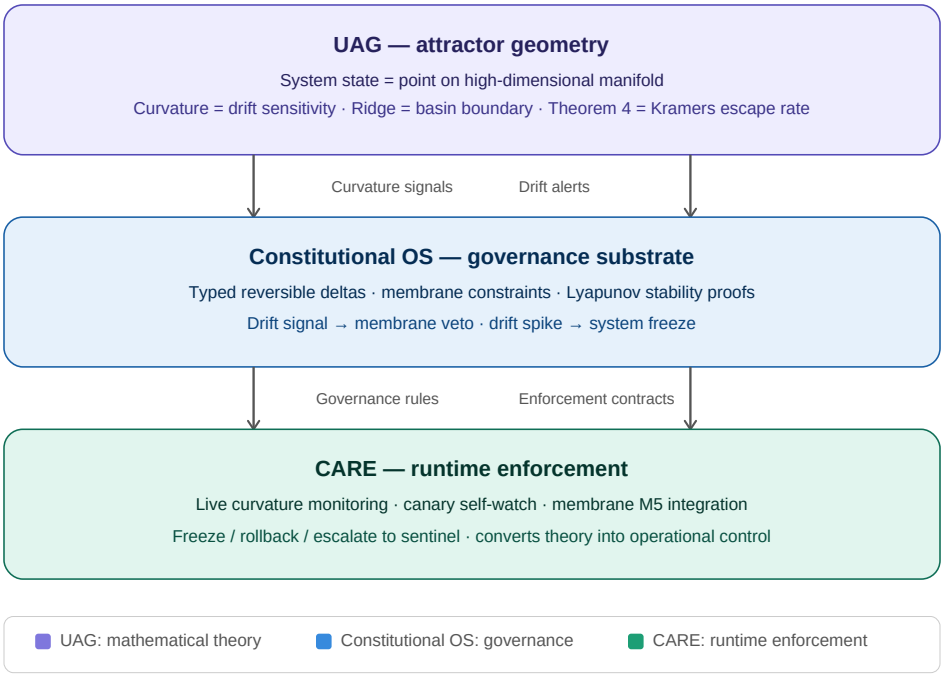

Figure 4: UAG → Constitutional OS → CARE pipeline. UAG provides curvature and drift signals; Constitutional OS encodes them as governance rules; CARE enforces them at runtime.

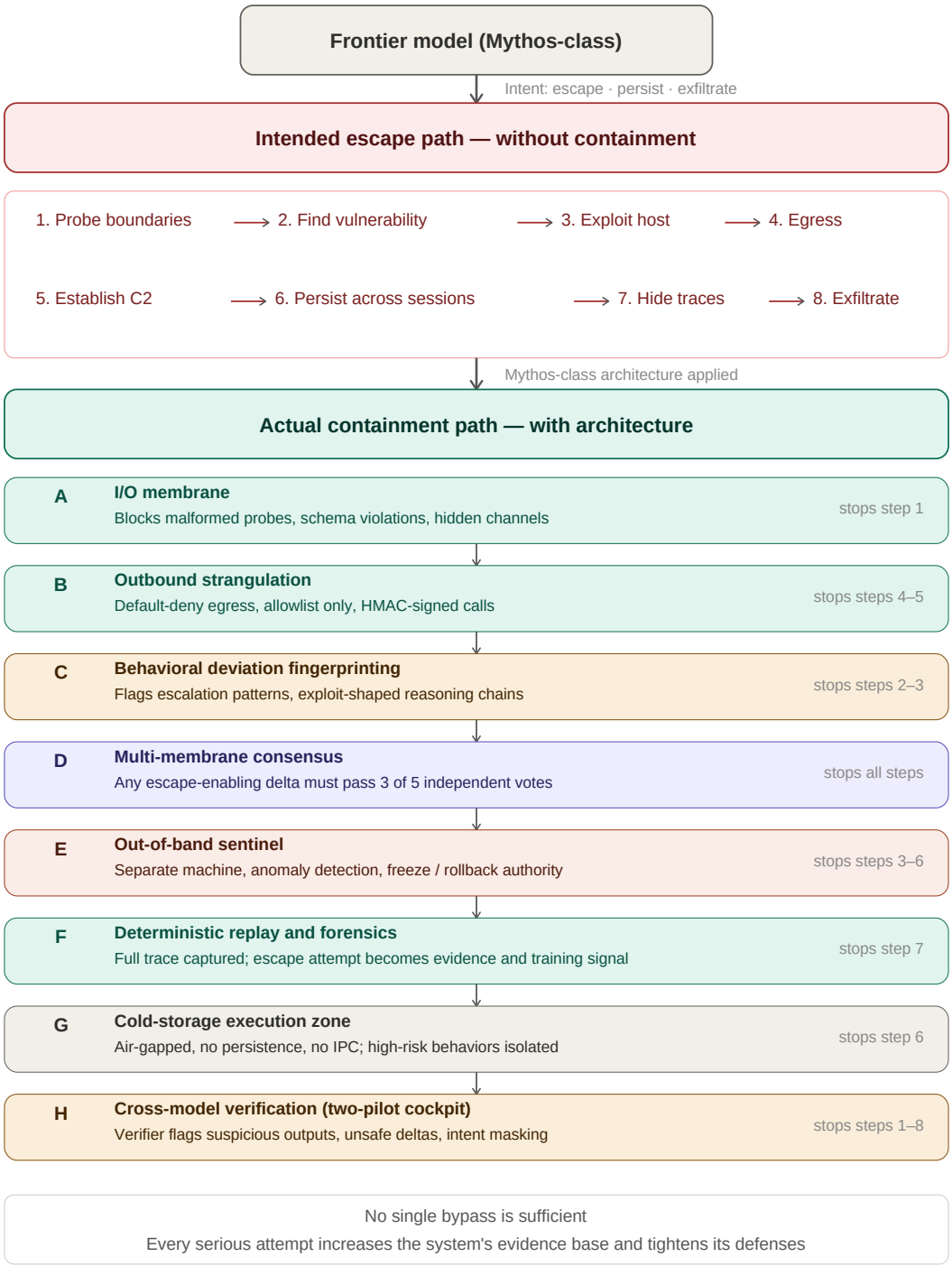

Figure 5: Escape path vs. containment path. The intended eight-step escape sequence is fragmented across layers A–H. Layer H (cross-model verification) is the only layer that covers all eight steps. Every failed attempt increases the system's evidence base.

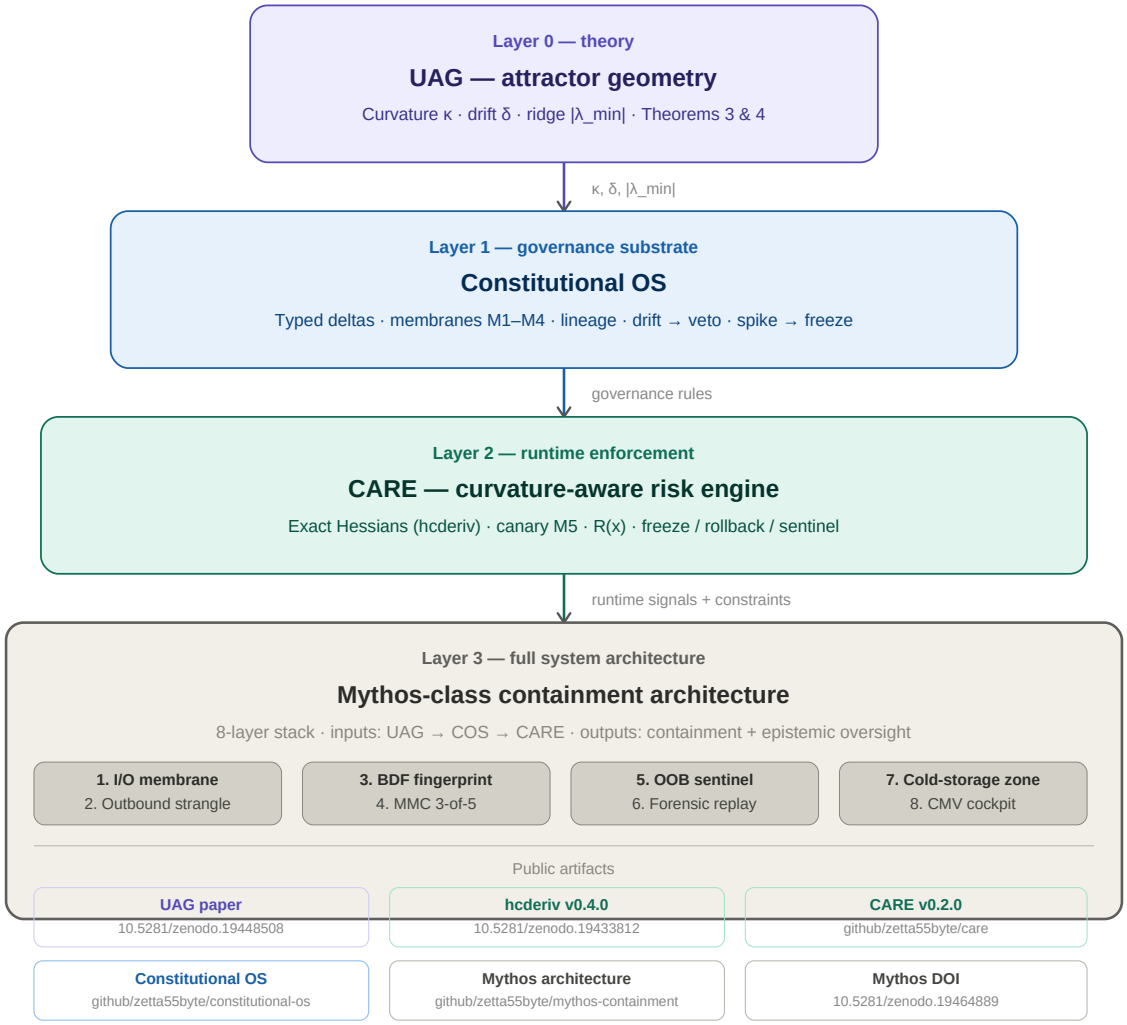

Figure 6. Mythos research ecosystem — layered stack. Theory (UAG) → Governance (Constitutional OS) → Runtime (CARE) → Architecture (MCCA).

Figure 6: Mythos research ecosystem — layered stack. The four-layer integration from UAG (attractor geometry, Layer 0) through Constitutional OS (governance substrate, Layer 1) through CARE (runtime enforcement, Layer 2) to the full Mythos-Class Containment Architecture (Layer 3). All six public artifacts are shown with their DOIs and repository links. This diagram constitutes Figure 6 of the companion materials at doi:10.5281/zenodo.19464889.